Here is all the code for this assignment - math796hw3.ipynb

Part 2) For the simple toy dataset referenced in the code for the paper, I implemented a per-sample line search with the armijo condition combined with a stochastic gradient descent for the descent direction. For BFGS, the implementation took in the full dataset for both the gradient descent and the line search. What I found here was that both the SGD with the fixed learning rate, and the BFGS were able to reach 100% accuracy while the SGD with line search misclassified one data point resulting in a 90% accuracy. I chose full batch for BFGS due to its nature of needing deterministic gradients and the fact that the noise in stochastic methods can corrupt the Hessian approximations.

It was a little interesting to see that SGD with line search could not achieve 100% accuracy on such a simple data set composed of only 10 points. This may be due to the noise of the single sample and that satisfying the armijo condition shrinks the learning rate down too low.

I also found that SGD with fixed learning rate was a lot faster computationally than SGD with line search and especially the BFGS which is to be expected.

Part 3) I chose the scipy global optimizers: differential evolution and dual annealing and basin hopping + l-bfgs.
These were the results I found:

```
=== Comparison (lower cost is better) ===
Method                                 Cost      Acc    Time(s)
SGD (η=0.75)                       0.000360    1.000      11.59
Differential Evolution             0.575030    1.000      15.56
Dual Annealing                     0.745319    1.000       5.13
Basin-Hopping + L-BFGS-B           0.290517    1.000      34.65

Best: SGD (η=0.75) | Cost=0.000360 | Acc=1.000 | Time=11.59s
```

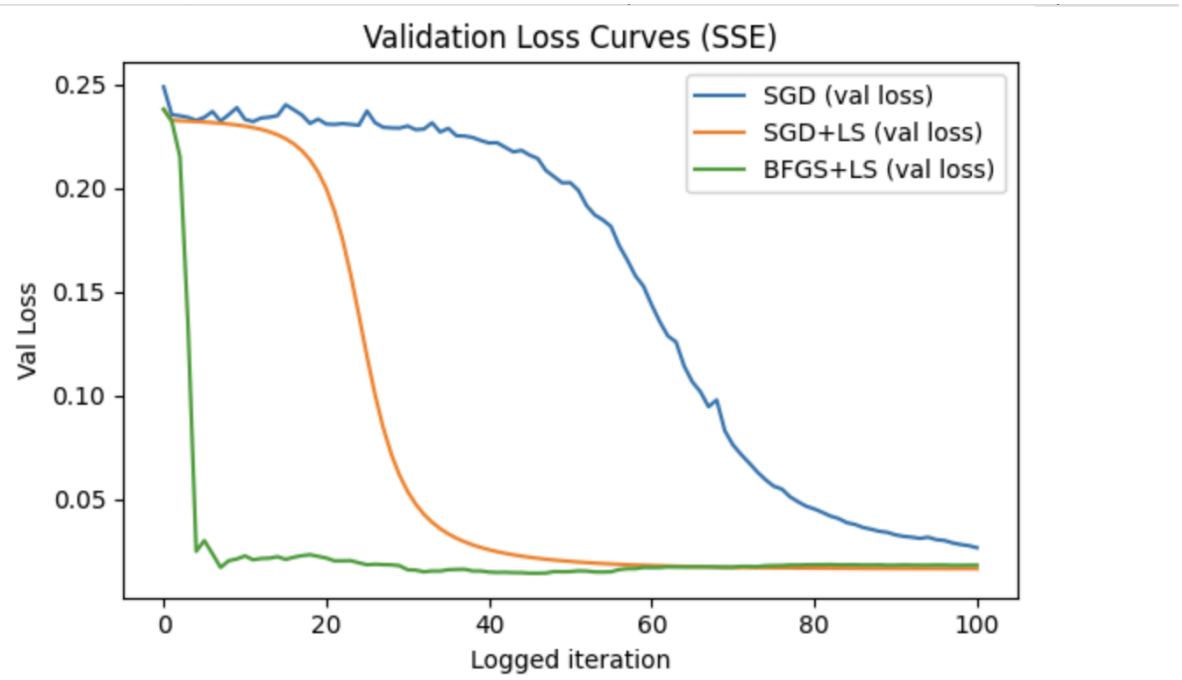
All the global optimizers were able to achieve 100% accuracy which is to be expected. However, it was interesting to see how much better the original code with the simple sgd and fixed learning rate did better than the global optimizers in terms of reducing the cost function. The sgd was much more confident in its classifications than the global optimizers. It was also interesting to notice how quickly dual annealing converged, even faster than the SGD but that was probably because again how simple the problem was and the fact that the SGD had a fixed number of iterations (100,000) while dual annealing did not. Also basin hopping had significantly less total cost than the other two global optimizers but it also took significantly longer to converge.

Part 4) The kaggle dataset I chose was the Wisconsin Breast Cancer dataset - https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data .
The reason I chose this what this was while it was a lot more complex than the toy dataset in the original code (dataset had 30 features, and 569 samples), it still was not too complex that the global optimizers would take too long to converge. Also the dataset was composed of

entirely numerical features and was a 2 class classification similar to the toy dataset so no preprocessing of the data had to be done.

Here are the results I got-

```
=== Optimizer Comparison (Breast Cancer, loss = SSE) ===
Method                          Time(s) |   TrainL     ValL    TestL |  TrainA    ValA   TestA
SGD (eta=0.05, sse)                1.40 |   0.0291   0.0250   0.0392 |   0.979   0.974   0.947
SGD + Armijo LS (sse)              1.24 |   0.0134   0.0140   0.0299 |   0.988   0.974   0.974
BFGS + Armijo LS (sse)            12.32 |   0.0040   0.0129   0.0405 |   0.997   0.982   0.939
Differential Evolution (sse)     233.94 |   0.0337   0.0323   0.0458 |   0.971   0.956   0.965
Dual Annealing (sse)             123.66 |   0.0044   0.0105   0.0397 |   0.997   0.974   0.930
Basin-Hopping + L-BFGS-B (sse)   475.20 |   0.0041   0.0122   0.0406 |   0.997   0.982   0.939
```



Here I found that SGD + LS had the highest test accuracy and it also had the least training time showing its effectiveness for this dataset. BFGS + line search, dual annealing and basin hopping all showed near-perfect accuracy for the test dataset and minimized training loss to nearly 0, but the accuracy significantly dipped for the test set, showing clear overfitting.

The global optimizers also took significantly longer to converge than the local methods, and also did not really offer any improvement in accuracy over the faster methods (they actually were significantly less accurate than the local methods due to overfitting except for differential evolution). Also, even though differential evolution achieved decent test accuracy, its losses were still higher indicating less confident predictions.

Overall this shows the effectiveness of stochastic gradient descent coupled with adaptive learning rate implemented with line search (adaptive gradient based methods more generally), both in terms of testing accuracy and computational efficiency, at least for this particular kaggle dataset.